

SYSTEM DESIGN

PHASE 8

SHARDING / PARTITIONING

High-End Interview Notes • Beginner-Friendly • Practical

Core Idea

When one database becomes too large or too busy for one machine, partition the data across multiple database nodes. Each piece is a shard/partition.

Memory Line

Sharding = SPLIT DATA. Shard key = DECIDE WHERE. Rebalancing = MOVE DATA. Hot partition = UNEVEN LOAD.

0. Learning Map

By the end of this phase, you should be able to explain why sharding exists, choose a shard key, compare partitioning strategies, and reason about distributed query/transaction costs.

Topic	Meaning	Importance
Sharding	Split one logical dataset across multiple database nodes	★★★★★
Shard key	Field used to decide the shard	★★★★★
Hash-based	Hash key → shard	★★★★★
Range-based	Value range → shard	★★★★★
Geographic	Region/location → shard	★★★
Hot partitions	One shard gets too much traffic	★★★★★
Rebalancing	Redistribute data after topology changes	★★★★
Cross-shard queries	One query touches multiple shards	★★★★★
Cross-shard transactions	One transaction spans shards	★★★★★
Consistent hashing	Reduce remapping when nodes change	★★★★

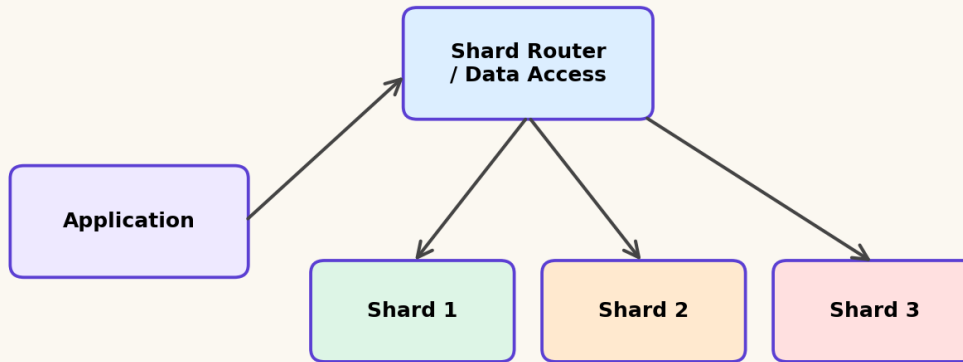
Prerequisite

Before sharding, understand indexes, transactions, replication, caching, load balancing, and basic distributed systems. Sharding is powerful—but it adds routing and operational complexity.

1. What Is Sharding?

Sharding means splitting a large logical dataset into smaller pieces and storing those pieces on different database nodes.

Sharding: One Logical Database → Multiple Physical Shards



Simple example

```

10 million users
↓
Shard by user_id
↓
Shard 1 → subset of users
Shard 2 → subset of users
Shard 3 → subset of users
  
```

- Without sharding: one database machine may become the storage, CPU, memory, and I/O bottleneck.
- With sharding: storage and query load can be spread across multiple machines.
- Sharding is usually a horizontal scaling technique for the data layer.
- Replication and sharding solve different problems: replication copies data; sharding divides data.

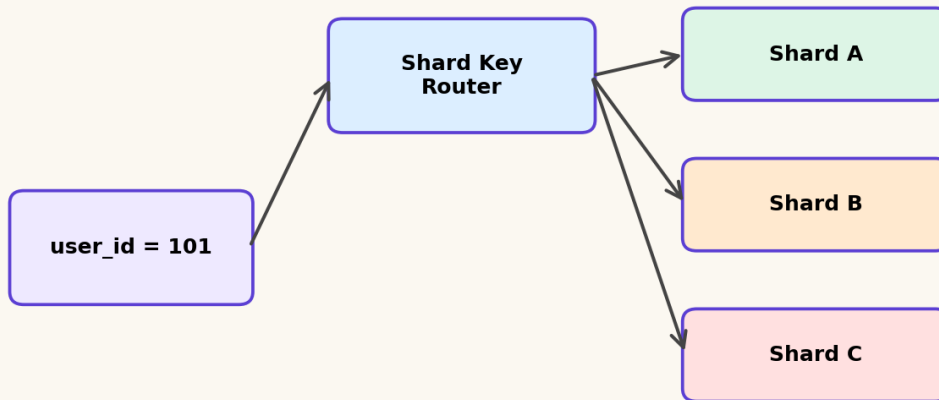
Interview sentence

“Sharding horizontally partitions a logical dataset across multiple database nodes so storage and workload can scale beyond a single node.”

2. Shard Key

The shard key is the field—or combination of fields—the system uses to determine which shard owns a record.

Shard Key Determines Where a Row Lives



A good shard key usually has

- High cardinality: many distinct values.
- Good distribution: avoids concentrating most records on one shard.
- Stable values: changing the key should not frequently move data.
- Query alignment: common queries should be able to identify the target shard.
- Acceptable write distribution: avoid sending most writes to one shard.

⚠ Biggest design mistake

Choosing a shard key only because it is easy. A technically valid key can still create hot partitions or force expensive scatter-gather queries.

3. Hash-Based Sharding

Hash-based sharding applies a hash function to the shard key and uses the result to select a shard.

```
shard = hash(user_id) % N
```

```
user_id = 101
hash(101) → 7
7 % 4 → shard 3
```

Advantage	Cost / Risk
Usually distributes keys evenly	Range queries become difficult
Simple routing	Changing N can remap many keys
Good for point lookups by key	Poor locality for ordered data
Can reduce sequential hotspots	Cross-shard aggregation may still occur

When to use

- Workloads dominated by point lookups.
- You want relatively even distribution.
- Queries rarely ask for contiguous ranges across the shard key.

Interview tip

Hash sharding is often a strong choice when even distribution matters more than range locality.

4. Range-Based Sharding

Range-based sharding assigns contiguous value ranges to different shards.

```
Shard A → user_id 1–1,000,000
Shard B → user_id 1,000,001–2,000,000
Shard C → user_id 2,000,001–3,000,000
```

Strength

- Excellent locality for range queries.
- Easy to understand and reason about.
- Useful for time-based data such as logs or events.

Weakness

- Sequential keys can make the newest range receive most writes.
- Ranges may become uneven in size or traffic.
- Rebalancing may be required as ranges grow.

Classic hotspot

If new records always have increasing IDs/timestamps, the newest range may receive nearly all writes. This is a hot partition problem.

5. Geographic Partitioning

Geographic partitioning places data according to geography, such as country, region, or continent.

```
region = user.region
```

```
India   → Asia shard(s)
Germany → Europe shard(s)
USA     → North America shard(s)
```

- Can reduce latency by keeping data near users.
- Can help with data residency or jurisdictional requirements when the design and laws require regional isolation.
- Works naturally when most requests are region-local.
- Can become unbalanced if one region is much larger or more active than another.

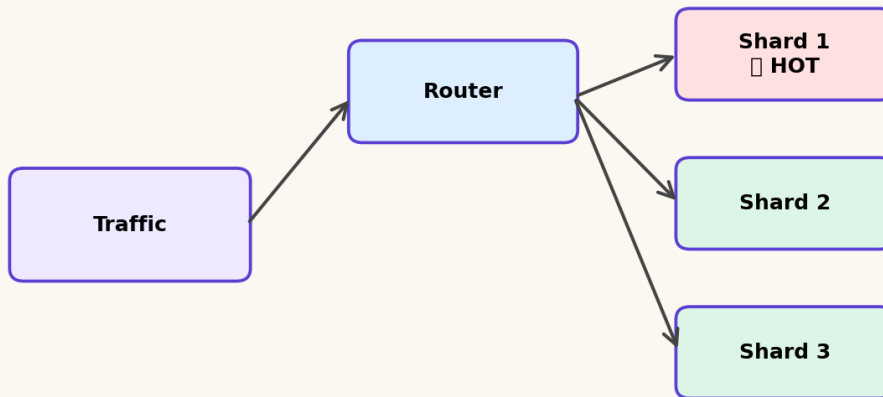
Important

Geographic partitioning is a data-placement strategy. It does not automatically guarantee legal compliance; actual requirements depend on the jurisdiction, data type, and architecture.

6. Hot Partitions

A hot partition happens when one shard receives disproportionately more traffic, storage, or writes than the others.

Hot Partition: One Shard Receives Disproportionate Traffic



Common causes

- Low-cardinality shard key.
- Sequential or predictable keys with range sharding.
- One celebrity/tenant/account generating huge traffic.
- Geographic imbalance.
- A popular object or key concentrated on one shard.

Ways to reduce hotspots

- Choose a better shard key.
- Use a composite key where appropriate.
- Add controlled key spreading/suffixing for extremely hot entities.
- Split or move hot ranges.
- Cache extremely popular reads.
- Use a strategy that distributes writes rather than concentrating them.

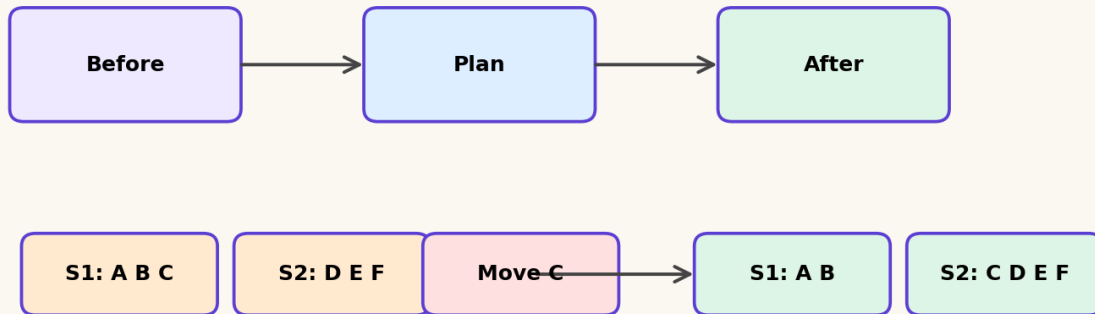
Interview answer

“I would first identify whether the hotspot is caused by the shard key, access pattern, or a specific tenant/entity. Then I’d change the partitioning strategy or split/spread the hot workload rather than simply adding more hardware.”

7. Rebalancing

Rebalancing means redistributing partitions or shards when nodes are added, removed, or become unevenly loaded.

Rebalancing: Moving Partitions Without Losing Data



- New shard added → existing data may need redistribution.
- One shard becomes much larger than others.
- A node is removed or replaced.
- Traffic distribution changes over time.

Good rebalancing goals

- Minimize downtime.
- Move data safely and incrementally.
- Keep reads/writes available where possible.
- Verify data before deleting the old copy.
- Throttle migration so rebalancing does not overload production.

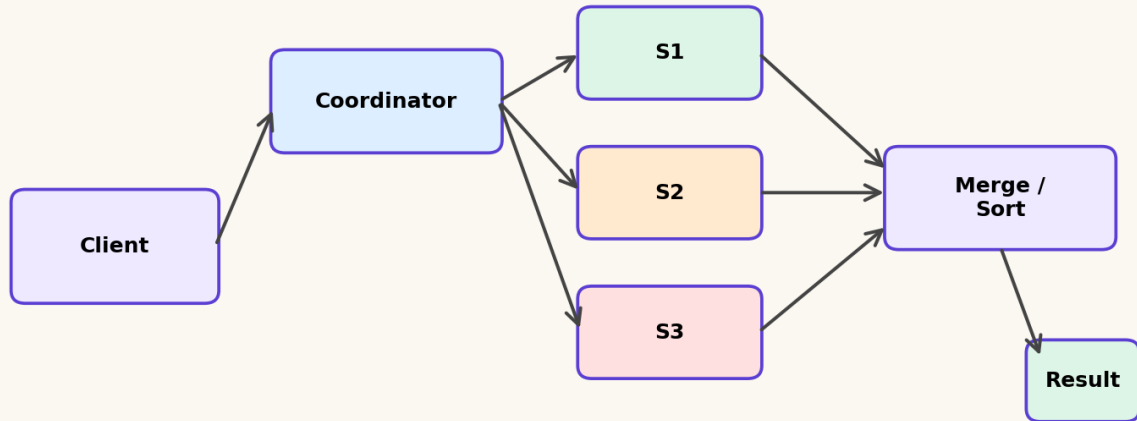
Think of it like this

Sharding is the initial seating plan. Rebalancing is moving people to different tables when the restaurant gets busier.

8. Cross-Shard Queries

A cross-shard query touches more than one shard because the system cannot identify a single shard from the query conditions.

Cross-Shard Query: Scatter → Query → Gather → Merge



```

SELECT COUNT(*)
FROM orders
WHERE status = 'PAID';
  
```

If orders are sharded by `user_id`,
`status='PAID'` does not identify one shard.

- Query relevant shards
- Gather results
- Combine them

Scatter-Gather

- Scatter: send the query to multiple shards.
- Gather: collect partial results.
- Merge: combine, sort, or aggregate them.
- Cost grows with the number of shards touched.

Design principle

A good shard key should align with important query patterns so the majority of requests can be routed to one shard—or a small, predictable set of shards.

9. Cross-Shard Transactions

A cross-shard transaction modifies data that lives on multiple shards and therefore requires distributed coordination.

Why it is difficult

- Each shard has its own local transaction boundary.
- A failure can happen after one shard commits but before another does.
- Distributed commit protocols add latency and operational complexity.
- Retries can create duplicate effects unless operations are designed carefully.

Approach	Idea	Trade-off
Avoid	Keep a transaction within one shard	Simplest, but constrains data model
Saga	Local transactions + compensating actions	More application complexity

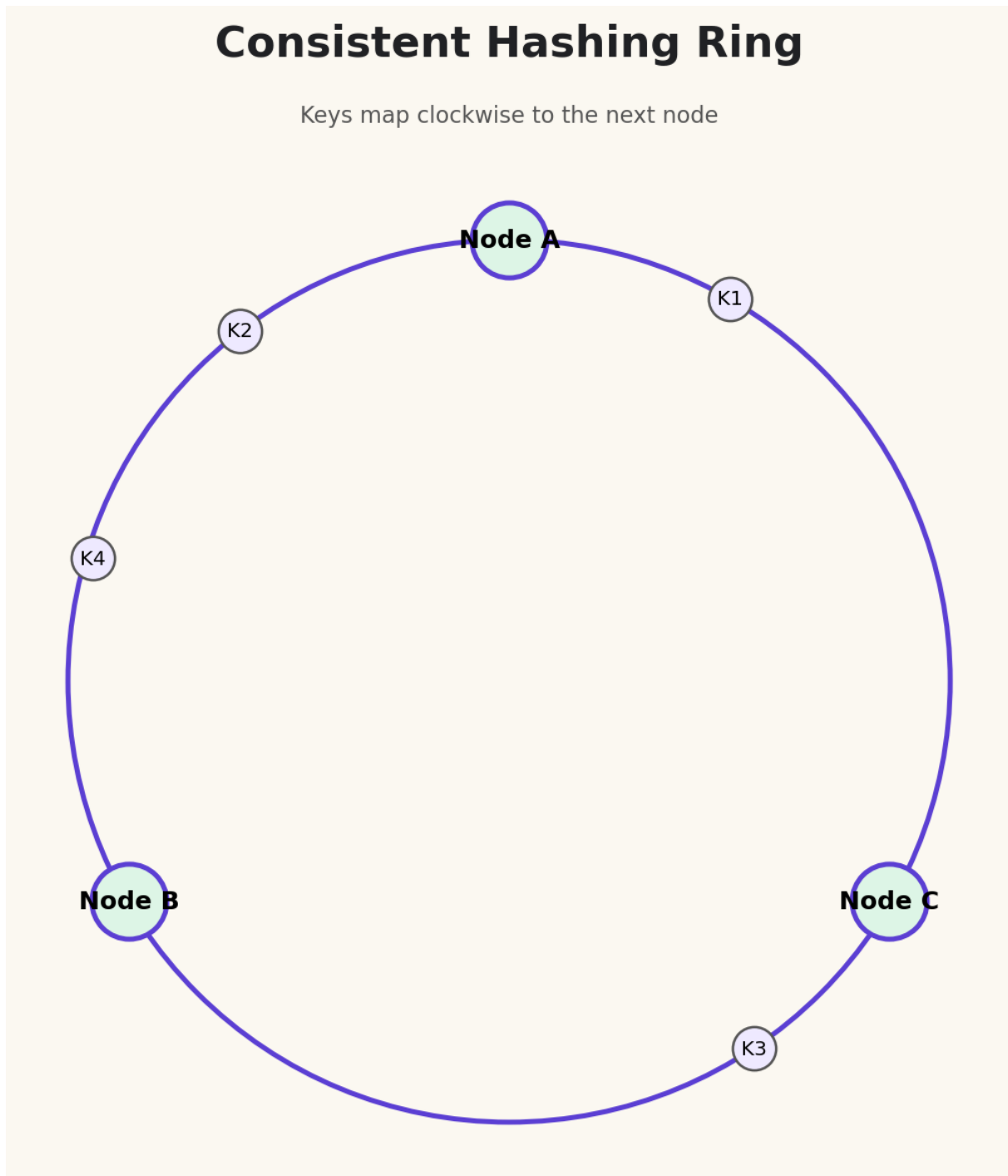
2PC / distributed transaction	Coordinate commit across participants	Higher coordination and failure complexity
Async/event-driven	Events + eventual reconciliation	Scalable, but not immediately consistent

Interview tip

When possible, design the shard key so an important business transaction is co-located on one shard. Avoid making distributed transactions the default.

10. Consistent Hashing

Consistent hashing maps keys and nodes onto a logical ring. When nodes change, ideally only a portion of keys need to move.



Why ordinary modulo can be painful

$\text{shard} = \text{hash}(\text{key}) \% N$

$N = 4 \rightarrow$ keys map to 0..3

$N = 5 \rightarrow$ many keys remap

$\rightarrow$ large cache/data movement

Consistent hashing idea

- Hash each node onto a ring.
- Hash each key onto the same ring.
- Walk clockwise to find the responsible node.
- Add/remove a node $\rightarrow$ only a portion of keys need remapping.
- Virtual nodes can improve distribution by placing multiple logical points per physical node.

Important nuance

Consistent hashing is especially associated with distributed caches and key-distribution systems. It can also be used in distributed data placement, but database sharding often uses other partition maps/routing mechanisms too.

11. Partitioning Strategy Comparison

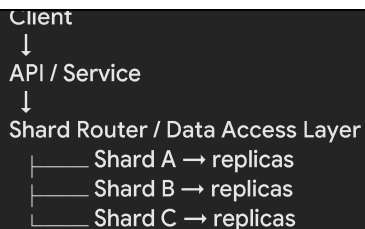
Strategy	Distribution	Range Queries	Hotspot Risk	Typical Fit
Hash	Usually good	Poor	Low-medium if key is good	Point lookups, even load
Range	Can be uneven	Excellent	Medium-high	Time/order/range workloads
Geographic	Region-dependent	Good for local region	Depends on region size	Latency/residency
Consistent hashing	Usually good	Poor	Can be controlled	Caches/distributed key placement

Remember

There is no universally best strategy. Choose based on access patterns, distribution, locality, operational constraints, and transaction boundaries.

12. How Sharding Fits Into a Real System

A production design commonly combines routing, replication, caching, and observability around sharded storage.



Cache may sit before the database.

Monitoring tracks shard size, latency, errors, and hot keys.

- Application/router determines the target shard from the shard key.
- Each shard can itself have replicas for availability/read scaling.
- Load balancing can exist inside or across database infrastructure.
- Metrics should be per-shard, not only global, so hotspots are visible.

Operational metrics to watch

Shard size • QPS per shard • write rate • p95/p99 latency • CPU • storage • replication lag • error rate • migration progress

13. Sharding Trade-offs

Benefit	Cost
Scales storage beyond one node	More operational complexity
Distributes reads/writes	Cross-shard queries become expensive
Can reduce single-node bottlenecks	Distributed transactions are difficult
Enables horizontal DB growth	Rebalancing/migrations are complex
Can improve regional locality	Bad keys can create hotspots

Rule of thumb

Do not shard just because “large systems use sharding.” First identify the bottleneck, estimate growth, and confirm that simpler scaling options are insufficient.

14. Interview Questions & Model Answers

Q1. What is sharding?

Sharding horizontally partitions a logical dataset across multiple database nodes so storage and workload can scale beyond one node.

Q2. What is a shard key?

It is the field or fields used to determine which shard stores a record. A good key distributes load and aligns with common query patterns.

Q3. Hash vs range sharding?

Hash generally provides better distribution; range provides better locality for ordered/range queries but can create hotspots.

Q4. What is a hot partition?

A partition that receives disproportionately high traffic or data, making that shard a bottleneck even when other shards are underused.

Q5. Why are cross-shard queries expensive?

They may require scatter-gather across multiple nodes, network calls, parallel execution, and result merging.

Q6. How do you avoid cross-shard transactions?

Choose a shard key that co-locates data involved in common transactions, or redesign the workflow around asynchronous/event-driven or saga-style coordination.

Q7. What is rebalancing?

Redistributing data among shards after topology or workload changes so capacity and traffic remain reasonably balanced.

Q8. What is consistent hashing?

A key/node mapping technique that reduces the number of keys that must move when nodes are added or removed.

Q9. Sharding vs replication?

Sharding splits data across nodes; replication copies data to multiple nodes. They are often combined.

Q10. What makes a bad shard key?

Low cardinality, skewed distribution, frequent changes, hotspot-prone values, or a key that causes common queries to scatter across many shards.

15. Mini System Design Exercise

Design the database layer for a social app with 100M users and very high traffic.

Step 1 — Identify the entity

Users, posts, comments, likes, follows.

Step 2 — Identify access patterns

Get user by ID; get posts for a user; get feed; write likes/comments; query recent posts.

Step 3 — Choose candidate shard keys

- Users → user_id is a natural candidate.
- Posts → author_id can co-locate a user's posts, but feed queries may span many authors.
- Time-only range partitioning may create a hot newest partition.

Step 4 — Identify distributed operations

- Home feed may need fan-out/read aggregation.
- Global counts may require distributed aggregation.
- Follow relationships may need careful data modeling to avoid expensive cross-shard joins.

Step 5 — State the trade-off

Strong interview framing

"I would start from access patterns, choose a shard key that keeps common operations local, measure distribution and hotspots, and introduce secondary strategies only where the workload requires them."

16. One-Page Revision Cheat Sheet

Concept	Remember
Sharding	Split one logical dataset across multiple nodes
Shard key	Determines where a record lives
Hash sharding	hash(key) → shard; good distribution
Range sharding	value range → shard; great locality
Geographic	region → shard; locality/residency use cases
Hot partition	One shard gets too much load
Rebalancing	Move/redistribute data as topology/load changes
Cross-shard query	Scatter → query → gather → merge
Cross-shard transaction	Distributed coordination; avoid when possible
Consistent hashing	Reduce key movement when nodes change
Replication	Copies data; different from sharding

Interview Memory Map

SHARDING = SPLIT DATA
 SHARD KEY = DECIDE WHERE
 HASH = EVEN DISTRIBUTION
 RANGE = GOOD LOCALITY
 GEO = KEEP DATA/TRAFFIC REGIONAL
 HOT PARTITION = UNEVEN LOAD
 REBALANCE = REDISTRIBUTE
 CROSS-SHARD QUERY = SCATTER + GATHER
 CROSS-SHARD TXN = HARD → AVOID/COORDINATE
 CONSISTENT HASHING = LESS REMAPPING

Before moving to the next phase, you should be able to answer

- Why would a database need sharding?
- How would you choose a shard key?
- When would you prefer hash vs range?
- What causes hot partitions?
- How would you handle a cross-shard query?
- Why are cross-shard transactions difficult?
- What is rebalancing?
- How does consistent hashing reduce movement?
- How is sharding different from replication?

17. Quick Glossary

Term	Meaning
Shard	One logical piece of a sharded dataset
Partition	A subset of data; terminology varies by database/system
Shard key	Field(s) used for data placement
Router	Component that maps requests/keys to shards
Hot shard	Shard under disproportionate load
Scatter-gather	Query multiple nodes, then merge results
Rebalancing	Redistribute partitions across nodes
Virtual node	Multiple logical hash-ring positions representing one physical node
Co-location	Keeping related records on the same shard
Skew	Uneven distribution of data or traffic